Predicting Online Shoppers' Purchase Intent: A Two-Phase Machine Learning Pipeline from Exploratory Analysis to Classification

Iris Hoxha · Ina Osmeni · Ina Shametaj

Department of Computer Science, University of New York Tirana, Tirana, Albania

Abstract This paper presents a complete two-phase machine learning pipeline for predicting purchase intent using the UCI Online Shoppers Purchasing Intention dataset, which covers 12,330 web sessions collected from a real e-commerce platform over one calendar year. The dataset presents a significant class imbalance challenge, with only 15.5 % of sessions resulting in a purchase, which influenced the design of every component of our pipeline. In Phase 1 we carried out exploratory data analysis, identified and handled missing or duplicate records, applied appropriate transformations to skewed features, engineered 15 new session-level variables motivated by the EDA findings, and selected the 20 most informative features through a two-stage collinearity filter and mutual information / random forest ranking process. We then produced a balanced training set using the Synthetic Minority Over-sampling Technique (SMOTE). In Phase 2, we trained and rigorously evaluated three classifiers of increasing complexity: Logistic Regression as an interpretable baseline, a Support Vector Machine with an RBF kernel to capture non-linear boundaries, and a feedforward Neural Network with Dropout regularisation and Batch Normalisation as the most flexible approach. All models were compared on the same held-out test set using six evaluation metrics, with F1-score and PR-AUC serving as primary criteria given the class imbalance. Our results show that the Neural Network achieves the best PR-AUC of 0.716, but Logistic Regression nearly matches it on F1-score (0.674 versus 0.669). We argue that this near-parity is a direct consequence of how thoroughly the Phase 1 feature engineering linearised the non-linear structure of the data, effectively reducing the advantage that more complex models would otherwise enjoy. SVM underperformed on all metrics, which we attribute to overfitting of its tight-margin configuration to the SMOTE-generated synthetic training examples.

Keywords: Purchase Intent Prediction, E-commerce Analytics, Class Imbalance, SMOTE, Feature Engineering, Logistic Regression, Support Vector Machine, Neural Network, Precision-Recall AUC, Exploratory Data Analysis.

1. INTRODUCTION

Online retail has grown substantially over the past decade, with global e-commerce revenue exceeding several trillion dollars annually. As platforms scale, they accumulate enormous amounts of session-level behavioural data: every click, every page view, every scroll is recorded in server logs or analytics systems. In principle, this wealth of data should make it easier to predict which visitors are likely to complete a purchase. In practice, the problem turns out to be considerably harder than it looks, and it has attracted sustained research attention from both academic machine learning and applied data science communities.

The core difficulty is not algorithmic but distributional. The vast majority of website visits do not end in a transaction. Industry estimates for e-commerce conversion rates typically range from 1 % to 5 %, varying significantly by product category, platform design, and the demographic of the visitor base. The dataset we worked with is somewhat higher, at 15.5 %, which reflects a reasonably engaged audience, but the fundamental problem remains: most visitors leave without buying anything. A classification model that simply predicts "no purchase" for every session will achieve 84.5 % accuracy without learning a single meaningful pattern. This makes raw accuracy a misleading optimisation target and requires us to think carefully about which metrics actually capture what we care about.

The commercial motivation for solving this problem is straightforward. If a platform can identify, in real time or near-real time, which visitors are likely to complete a purchase, it can target its interventions precisely. Common interventions include displaying a limited-time discount to hesitating buyers, opening a live chat window during moments of apparent deliberation, surfacing a specific product recommendation based on what pages the visitor has browsed, or sending a retargeting email after a session ends without a purchase. When interventions are applied broadly and indiscriminately, their cost-per-conversion rises substantially and the overall customer experience deteriorates because many recipients find the messaging irrelevant or intrusive. When they can be triggered selectively for high-probability buyers, resources are used more efficiently and the user experience for low-probability visitors is not unnecessarily disrupted.

The flip side of this argument also holds: a model that can reliably identify sessions that are very unlikely to convert allows the platform to deprioritise marketing budget for those users and allocate it toward sessions where the probability of conversion is higher. This dual application—targeting likely buyers and deprioritising unlikely ones—means that even modest improvements in classifier performance can translate into meaningful commercial impact at scale.

We worked with the Online Shoppers Purchasing Intention dataset published by Sakar et al. [2], available through the UCI Machine Learning Repository [12]. It covers 12,330 sessions from a single anonymous e-commerce website recorded over a full calendar year, spanning a wide variety of session types, visitor profiles, and product pages. Each session is represented by 17 features describing the pages visited, time spent, standard web analytics engagement metrics, contextual information about the timing and device, and visitor type. The target variable is binary: *Revenue* is 1 if the session ended with a completed purchase and 0 otherwise.

Our work is structured in two phases, which correspond to the two Jupyter notebooks submitted alongside this paper. Phase 1 covers the entire data understanding and preparation pipeline: exploratory data analysis of all 17 original features, cleaning and type conversion, feature transformation, construction of 15 new session-level features, two-stage feature selection, scaling, and SMOTE-based oversampling of the training set. Phase 2 covers the classification pipeline: training three models with systematic hyperparameter tuning, evaluating all three on the same held-out test set, and interpreting the results in terms of what they reveal about the structure of the problem.

We selected Logistic Regression, SVM, and a Neural Network because they represent three meaningfully different points in the complexity-interpretability trade-off. Logistic Regression is the simplest option, fully transparent, and fast to train and deploy. It gives us a lower bound on what careful feature engineering alone can achieve on this problem without any non-linear modelling capacity. SVM with an RBF kernel introduces non-linear decision boundaries at the cost of interpretability and significantly higher training time, occupying a middle ground in the expressiveness spectrum. The Neural Network is the most expressive of the three, capable in principle of learning arbitrary feature interactions through its multiple hidden layers, but the hardest to tune and the hardest to explain. Putting all three on the same held-out test set allows us to make evidence-based claims about whether added complexity pays off for this specific problem and dataset combination.

This comparison is not merely a performance exercise. The relationship between Phase 1 engineering and Phase 2 results tells a story about how data preparation interacts with model selection. We believe the main finding of this paper is less about which model wins and more about why the performance gaps are shaped the way they are.

The rest of this paper is organised as follows. Section 2 reviews prior work on purchase intent prediction, class imbalance handling, and the specific modelling techniques we used. Sections 3 and 4 describe the dataset and cover Phase 1 in full. Section 5 describes the Phase 2 modelling methodology. Section 6 presents all results. Sections 7 and 8 discuss the findings and conclude.

2. RELATED WORK

2.1 Purchase Intent Prediction

The problem of predicting purchase intent from session-level behavioural data has been studied for several decades, though the available data and the modelling tools have changed substantially over time. Early work tended to rely on simple decision rules or logistic regression applied to coarse-grained features like visit frequency or session duration. More recent work has access to richer, page-level clickstream data and a much broader set of modelling options ranging from gradient boosted trees to attention-based deep learning architectures.

The authors of our dataset, Sakar et al. [2], originally framed this as a real-time prediction problem and experimented with multilayer perceptrons and LSTM recurrent neural networks directly on the sequence of pages visited within each session. Their finding that tree-based methods with well-engineered session-level features performed competitively against more complex sequence models directly anticipated what we observed in our own results. The key insight from their work is that aggregating the sequence of page visits into session-level statistics—rather than trying to model the order of visits directly—retains most of the predictive signal while substantially reducing model complexity and training time. Their best system used a random forest on these aggregated features.

Moe [5] provided an early theoretical framework for distinguishing shoppers by intent based on in-store navigational clickstream data, identifying four distinct types of online shopping visits: directed buying, search/deliberation, hedonic browsing, and knowledge building. Each type produces a characteristic clickstream signature. A directed buyer visits product pages quickly and with high focus, spending little time on informational or administrative pages. A knowledge builder visits informational pages and leaves without committing to a purchase. Hedonic browsers visit many pages across many categories in a leisurely, exploratory way. Search/deliberation visitors compare specific products repeatedly before making a decision. This framework directly influenced how we thought about feature engineering in Phase 1: features like the ratio of product page visits to total pages, or time spent per page, are essentially trying to classify sessions by their position on Moe's spectrum of intent.

Sismeiro and Bucklin [11] modelled the probability of purchase completion as a function of observable session-level variables, finding that factors like the number of pages viewed, session duration, and whether the visitor had previously purchased were significant predictors. Critically, their work emphasised that much of the predictive information about purchase intent is already available before the session ends, which is the core premise of our problem setting. We do not need to model the sequence of events in real time; we can compute summary statistics at the end of the session and predict whether that session resulted in a

purchase.

More recent work has explored deep learning approaches. Entity embeddings for categorical variables, attention mechanisms for long browsing histories, and hybrid architectures combining clickstream sequences with session-level summaries have all shown improvements in specific settings. However, as several of these approaches require linking sessions to specific users across time to build behavioural histories, they are not directly applicable to our dataset, which contains no user identifiers. Each session in our data must be predicted in isolation, without any access to prior sessions by the same visitor.

2.2 Class Imbalance in Classification

Class imbalance is one of the most practically important challenges in applied machine learning, and it is absolutely central to our problem. The theoretical analysis by He and Garcia [3] remains the most comprehensive treatment of this topic. Their key argument is that standard performance metrics based on the confusion matrix, particularly accuracy, are fundamentally broken for imbalanced problems: a trivial classifier that always predicts the majority class will achieve accuracy equal to the majority class proportion while making zero useful predictions for the minority class. In our case, this trivial classifier would achieve 84.5 % accuracy while identifying exactly zero purchasing sessions. This is useless in practice but would look excellent under an accuracy-based optimisation criterion.

He and Garcia recommend precision, recall, F1-score, and area under the precision-recall curve as more meaningful metrics when the positive class is rare, because these metrics focus on how well the classifier identifies the minority class. They also discuss the difference between ROC-AUC and PR-AUC in detail. ROC-AUC uses the true negative rate on its x-axis. When the negative class is very large, as it is in imbalanced problems, even a poor classifier will achieve a high true negative rate, which compresses the ROC curve upward and makes all classifiers look good. PR-AUC avoids this by using precision, which is entirely determined by how many false positives the classifier produces relative to true positives, independent of the size of the negative class.

The two main approaches to handling class imbalance at the data level are undersampling the majority class and oversampling the minority class. Simple random undersampling discards a large fraction of the training data, which is wasteful and potentially throws away information that could help characterise the negative class boundary. Simple random oversampling duplicates minority-class examples without adding any new information, which can cause the model to overfit to those specific points because they are seen many times during training.

Chawla et al. [6] introduced SMOTE as a more principled oversampling alternative. Rather than duplicating existing minority-class examples, SMOTE creates new synthetic examples by linear interpolation between an existing minority-class point and one of its k nearest neighbours in feature space. For a given minority-class point $\mathbf{x}$, a neighbour $\mathbf{x}'$ is selected at random, and the synthetic point is $\mathbf{x} + \lambda(\mathbf{x}' - \mathbf{x})$ where λ is drawn uniformly from $[0, 1]$. This traces a line segment between two real minority-class examples and places the synthetic point at a random location along that segment. The resulting synthetic points fill in the local neighbourhood of the minority class rather than simply duplicating its existing members, producing a more diverse and typically better-calibrated training distribution.

A significant limitation we acknowledge in our work is that we applied SMOTE to the full training set before running cross-validation for hyperparameter tuning. The correct approach is to apply SMOTE inside a pipeline that confines oversampling to each training fold during CV. Because we applied it globally, the validation folds during CV contained synthetic examples that were generated from the same pool of real examples as the corresponding training fold. This is a known source of mild CV score inflation, often called the SMOTE leakage problem. The effect on hyperparameter selection is likely modest, but it is a methodological imperfection we would fix in a production implementation. Our test-set results are unaffected because the test set was never modified.

Beyond data-level approaches, algorithm-level approaches include adjusting the loss function to penalise minority-class errors more heavily. In scikit-learn, this corresponds to the `class_weight=balanced` option, which sets class weights inversely proportional to class frequencies. Combining both a data-level approach (SMOTE) and an algorithm-level approach (class weighting) can overcorrect the imbalance, which is likely why our best Logistic Regression configuration rejected class weighting when it had already been trained on SMOTE-balanced data.

2.3 Logistic Regression for Binary Classification

Logistic Regression is one of the oldest and most widely used classification methods. It models the log-odds of the positive class as a linear function of the input features. The appeal of the model is its interpretability: each coefficient directly represents the change in log-odds associated with a one-unit increase in the corresponding feature, holding all other features constant. For business applications where predictions need to be explained to non-technical stakeholders, or where auditing for fairness is required, this transparency is a genuine advantage that goes beyond raw performance numbers. It also makes the model easy to deploy in production: a logistic regression prediction is just a dot product followed by a sigmoid function, which can be computed in microseconds.

Regularisation is critical for logistic regression when features share some correlation, which is common in engineered feature sets. Without regularisation, the maximum likelihood solution may be poorly conditioned or even un-

defined when features are near-collinear. L1 regularisation (Lasso) adds a penalty term $\lambda \sum_j |\beta_j|$ to the loss, which shrinks some coefficients to exactly zero, performing implicit feature selection. L2 regularisation (Ridge) adds $\lambda \sum_j \beta_j^2$, which shrinks all coefficients proportionally toward zero, producing smoother and more stable solutions but retaining all features. The regularisation strength is controlled by the parameter C in scikit-learn's parameterisation, which is the inverse of the regularisation weight: a small C means strong regularisation and a large C approaches an unregularised model.

The choice between L1 and L2 often depends on whether feature selection is desirable. L1 produces sparse solutions, which can be useful when the number of features is large relative to the number of samples. L2 typically outperforms L1 when most features are genuinely informative, as was the case with our carefully selected 20-feature set.

2.4 Support Vector Machines

The Support Vector Machine, introduced by Vapnik [7], finds the hyperplane that separates the two classes with maximum margin. The key insight is that the decision boundary is determined solely by the training examples closest to it—the support vectors—making the model potentially robust to noise in the rest of the training distribution. The geometric intuition is that a wider margin means the decision boundary is less sensitive to small perturbations in the training data, which should translate into better generalisation.

The RBF kernel extends this idea to non-linear problems by implicitly mapping inputs into a higher-dimensional feature space where a linear separator may exist. The kernel function takes the form $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$, where γ controls the reach of each training point's influence. Large γ means each point influences only a small local region of the feature space; small γ means each point influences a broader area, making the boundary smoother.

The two key hyperparameters are C, which controls the trade-off between margin width and training error, and γ. A large C forces the model to minimise training errors even at the cost of a narrow margin, leading to more complex decision boundaries. A small C tolerates more training errors in exchange for a wider margin, which can improve generalisation. These two parameters interact: a high C with a large γ can produce highly irregular decision surfaces that closely track the training data without generalising well.

Fernández-Delgado et al. [8] conducted one of the largest empirical comparisons of classification methods, testing 179 classifiers on 121 datasets. RBF-SVMs were among the consistently top-performing methods across a wide range of problems, which motivated including it in our comparison. In practice, SVM's quadratic to cubic training complexity in the number of samples, $O(n^2)$ to $O(n^3)$ with the libsvm backend, makes it slow on larger datasets. With our 16,476-sample SMOTE training set, GridSearchCV with 3-fold CV already required several minutes, which is why we could not run it with 5-fold CV like Logistic Regression.

2.5 Neural Networks and Regularisation

Feedforward neural networks learn hierarchical non-linear representations of the input features through successive layers of learned transformations. For tabular classification problems like ours, they can in principle discover arbitrary feature interactions without those interactions being explicitly engineered by the modeller. The expressive power comes at a cost: more parameters mean more freedom to overfit, longer training times, more sensitivity to hyperparameter choices, and much less interpretability.

Two regularisation techniques are standard in modern feedforward networks. Dropout, introduced by Srivastava et al. [9], randomly sets a fraction of activations to zero during each training step, forcing the network to learn redundant representations and preventing individual neurons from becoming too specialised to specific training examples. The intuition is that any single neuron cannot rely on the presence of other specific neurons because those neurons may be randomly silenced during any training step. This has the mathematical effect of training an ensemble of exponentially many networks sharing weights, and is empirically effective at reducing overfitting across a wide range of tasks.

Batch Normalisation, introduced by Ioffe and Szegedy [10], normalises the distribution of activations across each mini-batch during training by subtracting the batch mean and dividing by the batch standard deviation, then applying a learned scale and shift. This stabilises the gradient flow through deep networks, allows higher learning rates to be used without the risk of divergence, and reduces sensitivity to the specific initialisation of weights. In combination with Dropout, it allows relatively deep networks to be trained on moderate-sized datasets without catastrophic overfitting, which was important for our training set of 16,476 samples.

For the specific problem of imbalanced binary classification, an important design choice is the early stopping criterion. It is tempting to monitor validation loss (binary cross-entropy), since that is what the network is directly minimising. However, on imbalanced data, the cross-entropy loss can continue decreasing even as the model shifts probability mass away from the minority class. The reason is that assigning very low probabilities to the many negative-class examples reduces loss more than correctly calibrating probabilities for the few positive-class examples. Monitoring validation AUC instead ties the early stopping criterion directly to the discriminative ability of the model across all thresholds, which is a better proxy for

the PR-AUC we ultimately care about.

2.6 Feature Engineering and Model Selection

There is a long-standing observation in the applied ML literature that the quality of features matters at least as much as the choice of model, particularly for structured tabular data. Domingos [1] argued directly that feature engineering is the single most important factor in the success of a learning algorithm, more important than the choice of algorithm itself. For tabular problems, deep learning has not displaced careful, domain-informed feature engineering in the way it has for images and text, because tabular features do not share the spatial or sequential structure that convolutional and recurrent architectures exploit.

Ngai et al. [4] reviewed over a decade of data mining applications in customer relationship management and found that preprocessing and feature engineering steps had a larger measured effect on final performance than the choice of classifier in the majority of surveyed papers. This is one of the most practically important findings in the applied ML literature for our domain, and it shaped how we prioritised our work: we put substantial effort into Phase 1 before writing a single line of modelling code in Phase 2.

The interaction between feature engineering and model selection is particularly relevant when comparing linear and non-linear models. A linear model can only draw straight hyperplanes through the input space. But if non-linear relationships between raw features and the target are explicitly encoded as new features—for example, by building a product of two features that captures a multiplicative interaction—then that non-linearity becomes a linear relationship between the new feature and the target. The linear model can then represent the interaction using a single coefficient on the new feature. This is the mechanism behind our key finding in Section 7.

2.7 Evaluation Metrics for Imbalanced Learning

Selecting the right evaluation metrics for an imbalanced classification problem is not a minor technical detail; it determines what the model is actually being optimised for and whether the results are meaningful. The standard metrics of accuracy, precision, recall, and F1-score describe performance at a single operating point defined by a chosen probability threshold, typically 0.5. Two additional metrics—ROC-AUC and PR-AUC—describe performance across all possible thresholds, summarising the entire operating characteristic of the classifier as a single number.

The area under the ROC curve (ROC-AUC) is the probability that a randomly chosen positive example receives a higher predicted probability than a randomly chosen negative example. For a completely random classifier, this is 0.5; for a perfect classifier, it is 1.0. ROC-AUC is a widely reported metric and enables comparisons across many published papers. However, as He and Garcia [3] noted, the ROC curve can be overly optimistic on imbalanced datasets because the false positive rate on the x-axis is computed relative to the very large number of true negatives, making it easy for all classifiers to maintain a low false positive rate even when they produce many false positives in absolute terms.

PR-AUC measures the area under the precision-recall curve, where precision and recall are computed at each possible threshold. A high PR-AUC means the classifier maintains high precision (few false positives among predictions) even while achieving high recall (identifying most true positives). This is harder to achieve on an imbalanced dataset because as recall increases, you eventually need to lower the threshold to the point where many negative examples start being predicted positive, causing precision to drop. PR-AUC is immune to the distortion that inflates ROC-AUC on imbalanced data, and it is generally considered the more reliable metric for cases where the positive class is rare. We used both metrics but treated PR-AUC as the primary summary metric for comparing calibration quality across our three models.

3. DATASET AND EXPLORATORY DATA ANALYSIS

3.1 Dataset Description

The Online Shoppers Purchasing Intention dataset [2, 12] was collected from a real e-commerce website over the course of a full calendar year. It contains 12,330 records, each representing a unique browsing session by an anonymous user. No information is available to link sessions to specific users, so each record must be treated as an independent observation. The data was originally collected and published as part of a study on real-time purchase intent prediction, and it has since become a widely used benchmark in the e-commerce classification literature due to its realistic imbalance ratio and the diversity of feature types it contains.

The 17 original features fall into three natural groups. *Page interaction features* describe the structure of the session in terms of page types visited: Administrative, Informational, and ProductRelated record how many pages of each type were visited, while Administrative_Duration, Informational_Duration, and ProductRelated_Duration record the total time spent on each type in seconds. These six features together provide a basic picture of both the breadth (how many pages) and depth (how much time) of the session, capturing two distinct aspects of engagement that are not perfectly correlated.

Engagement quality features capture how the visitor interacted with the content. BounceRates records the percentage of sessions that entered the site from a given page and immediately left without any further interaction; it is a standard Google Analytics metric for measuring whether content is relevant to the visitor's intent. ExitRates records the percentage of page views from a given page that were the last in the session. PageValues is derived from Google Analytics and represents the estimated monetary value of

a page, computed as a weighted average of transaction values for all sessions that included that page and ended in a purchase. Pages that appear frequently in conversion paths receive higher values, making PageValues a forward-looking proxy for how close a session is to the purchase funnel.

Contextual and demographic features include Month (the calendar month of the session), SpecialDay (a continuous score from 0 to 1 reflecting proximity to a commercial holiday), Weekend (a binary flag), OperatingSystems, Browser, Region, and TrafficType (all categorical identifiers), and VisitorType, which distinguishes Returning Visitors from New Visitors with a small third "Other" category.

Table 1 summarises the basic statistics for the key numerical features, split by the target class. The differences in means between the two classes are already visible at this stage and motivated several of the feature engineering choices described in Section 4.

Table 1: Summary statistics for key numerical features, split by purchase outcome.

Feature	No Purchase (mean)	Purchase (mean)
ProductRelated	28.4	46.0
ProductRel. Duration	1131	2084
BounceRates	0.023	0.010
ExitRates	0.044	0.028
PageValues	2.6	31.7
Administrative	2.2	2.8
Informational	0.54	0.61

3.2 Class Imbalance

Of the 12,330 sessions, 10,422 (84.5 %) did not result in a purchase, and 1,908 (15.5 %) did. The negative-to-positive ratio is approximately 5.46:1. This degree of imbalance is entirely expected for e-commerce data and is in fact mild compared to many fraud detection or medical diagnosis problems, where ratios of 100:1 or higher are common. Nevertheless, it is severe enough to make accuracy meaningless as a performance metric and to require deliberate strategies during both model training and evaluation.

We confirmed the class distribution at the very beginning of Phase 1 and treated it as a non-negotiable constraint throughout the entire pipeline. Every decision about how to split the data, which metrics to report during CV, how to tune hyperparameters, and how to handle the training data was made with this imbalance in mind. Setting up the evaluation framework before starting any modelling work is essential precisely because the alternative—choosing evaluation criteria after seeing initial results—creates opportunities for inadvertent result shopping.

3.3 Correlation Analysis

The most notable pairwise relationship in the dataset is between BounceRates and ExitRates, which have a Pearson correlation of approximately 0.91. Both features measure forms of disengagement, and they tend to move together because high-exit-rate pages often also see many users bouncing. For a Logistic Regression model, collinearity at this level inflates the variance of both coefficient estimates, making the model less stable across different training samples and reducing the interpretability of the individual coefficients. It also does not add useful information for non-linear models, since both features are measuring highly overlapping constructs.

We addressed this directly: we retained only the feature with the higher individual correlation with Revenue (ExitRates) and dropped BounceRates. This is a principled choice rather than an arbitrary one because ExitRates encodes a slightly different signal than BounceRates. BounceRates specifically measures sessions that enter from a page and immediately exit; ExitRates measures all page views that were the last of their session. The latter is the broader and more general measure of disengagement across the full session, which makes it the more informative of the two for our prediction task.

Among the page-type features, Administrative, Informational, and ProductRelated page counts are all moderately correlated with their corresponding duration features at approximately 0.5 to 0.7. This is expected: more pages visited generally means more time spent. These correlations are not high enough to trigger our collinearity threshold, and the count and duration features encode meaningfully different aspects of session depth.

PageValues stands out as the most individually predictive feature. Its point-biserial correlation with the binary Revenue target is above 0.5, substantially higher than any other original feature. The correlation between Administrative and Informational features and Revenue is comparatively low, below 0.1 in most cases. Visiting administrative or informational pages is not strongly associated with purchase intent, which is intuitively sensible: someone checking a return policy may or may not be about to buy, while someone spending time on high-PageValues product pages is much more likely to be in the purchase funnel.

3.4 Feature Distributions

The numerical features in this dataset are uniformly right-skewed, and in several cases severely so. ProductRelated page counts have a median of around 18 visits but a mean of approximately 31, driven by a small number of sessions with hundreds of product page visits. ProductRelated_Duration shows the same pattern, with a median around 1,000 seconds but a mean closer to 1,600, again pulled upward by a small number of extremely long sessions. These extreme values are unlikely to represent normal human browsing behaviour. A person visiting 300

product pages in a single session is far more likely to be an automated price scraper or comparison bot than a genuine customer deliberating over purchases.

Administrative and Informational features are even more extreme in their distribution: a large majority of sessions show exactly zero visits to these page types. For Administrative pages, approximately 65 % of sessions have zero visits; for Informational pages, the fraction is even higher at around 75 %. Most users go directly to product pages without visiting informational or administrative sections. This zero-inflation is not a data quality problem; it is a real and informative signal. A session that never visits an administrative page is qualitatively different from one that does, and this binary distinction is arguably more informative than the exact number of administrative pages visited. We needed to preserve this distinction while also handling the long right tail of the non-zero values.

PageValues has arguably the most striking distributional difference between classes. For sessions that did not result in a purchase, PageValues is heavily concentrated near zero with a very long but thin tail, reflecting the fact that most non-purchasing sessions stayed on pages that do not appear in conversion paths. For purchasing sessions, the distribution is much broader, extending into higher values, and the mean is nearly 12 times higher (31.7 versus 2.6, as shown in Table 1). This separation is clean enough that PageValues alone provides substantial predictive power, and it directly motivated several of our engineered features.

The BounceRates and ExitRates distributions are both concentrated near zero but not at zero: most sessions have some level of engagement, but a long tail of highly disengaged sessions exists. The difference between purchasing and non-purchasing sessions is consistent: buyers have lower BounceRates (0.010 versus 0.023) and lower ExitRates (0.028 versus 0.044), both reflecting greater engagement and less disengagement during the browsing session.

3.5 Temporal Patterns

The Month feature shows a clear seasonal pattern that was one of the most interesting findings in our EDA. November stands out by far as the highest-volume month, with both the most sessions and the highest purchase rate. This aligns directly with Black Friday, Cyber Monday, and the broader holiday shopping season. December is also elevated but less dramatically so, consistent with continued holiday shopping that may be somewhat more planful and less impulse-driven than the Black Friday period.

February and March show moderate increases in both traffic and purchase rates, consistent with Valentine's Day and early spring shopping patterns. May also shows a mild bump. The summer months—June through August—are noticeably quieter on both dimensions. July and August have among the lowest purchase rates of any month in the dataset, which aligns with summer browsing patterns in European markets where e-commerce tends to slow during vacation periods.

These seasonal patterns meant that Month, as a raw integer from 2 to 12, was not the most informative representation of the temporal signal. The month number encodes calendar position rather than commercial significance. A month number of 11 (November) and 12 (December) should be treated similarly for our purposes; the distinction between February (2) and September (9) is not as commercially meaningful as the distinction between the holiday period and the rest of the year. Grouping months into a Season variable was a more natural encoding of the underlying calendar structure.

The Weekend flag shows a smaller but consistent pattern. Weekday sessions convert at a marginally higher rate than weekend sessions across all months. One interpretation is that weekday browsers tend to have more specific, goal-directed shopping intent, arriving at the site with a clearer sense of what they are looking for. Weekend visitors may be in a more exploratory, leisure mode, browsing casually without a strong purchase intention. The effect is modest and not dominant on its own, but we noticed during EDA that returning visitors on weekends showed particularly high conversion rates relative to returning visitors on weekdays, which suggested an interaction term was worth constructing.

3.6 Categorical Feature Patterns

VisitorType is one of the more informative categorical features. Returning visitors convert at a meaningfully higher rate than new visitors: in our dataset, approximately 19.8 % of returning visitor sessions result in a purchase versus about 13.3 % of new visitor sessions. This difference makes intuitive sense. A returning visitor has already demonstrated enough interest in the platform to come back, and many of them may be returning specifically to complete a purchase they considered but did not commit to during a previous session. New visitors are more likely to be in exploratory mode, assessing whether the platform meets their needs before committing to any transaction.

The "Other" category in VisitorType accounts for fewer than 1 % of sessions and shows behavioural patterns more similar to new visitors than returning ones in our EDA, including similar PageValues distributions and similar BounceRate profiles. We merged it with New Visitor during cleaning, simplifying VisitorType to a binary IsReturning indicator.

TrafficType shows considerable variation in conversion rates across its categories, ranging from very high conversion rates in some types (likely direct navigation and branded search, where visitors arrive with specific intent) to low rates in others (likely display advertising traffic, where visitors may have clicked on a banner without strong intent). Because many TrafficType categories have small sample sizes, we grouped the long tail of rare types to avoid the model learning spurious patterns from insuffi-

cient examples.

SpecialDay is largely zero across the dataset. Most sessions fall on non-holiday days, and the SpecialDay score is non-zero only for sessions recorded close to Valentine's Day, Mother's Day, or similar commercial milestones. Where it is non-zero, there is a modest positive association with purchase probability, consistent with the idea that visitors near a major commercial holiday have somewhat stronger purchase intent. The signal is real but subtle and does not rival the predictive power of PageValues or the product page engagement features.

4. PHASE 1: DATA PREPARATION AND FEATURE ENGINEERING

4.1 Data Cleaning

The first step after loading the raw CSV was a systematic check for data quality issues. We found 125 fully duplicated rows and removed them, reducing the dataset from 12,330 to 12,205 unique sessions. Duplicate rows in clickstream data typically arise from logging errors, session tracking artifacts, or repeated page loads being counted as separate sessions rather than genuine repeated browsing behaviour. There was no reason to keep them; retaining duplicates would give those specific sessions artificially inflated weight during model training.

There were no missing values in any column of the cleaned dataset, which simplified the pipeline considerably. We did not need to make any imputation decisions. This is consistent with the dataset having been cleaned prior to publication, which is common for benchmark datasets hosted on the UCI repository.

Several columns required type conversions. Revenue and Weekend were stored as Boolean strings ("True"/"False") in the raw CSV and were converted to binary integers (0/1). Month was stored as three-letter abbreviations ("Feb", "Mar", ...) and was mapped to numeric values from 2 to 12 (February through December). January is absent from the dataset entirely: the original data collection period had no sessions from January, which we confirmed is consistent with the monthly distributions reported in Sakar et al. [2].

For OperatingSystems, Browser, and TrafficType, which are nominal categorical features with varying numbers of levels, we merged any level appearing in fewer than 1 % of sessions into a single "other" category encoded as -1. This prevents models from learning spurious patterns from very small subgroups that are too small to generalise reliably. For VisitorType, we simplified from three categories to a binary IsReturning flag by merging "Other" with "New Visitor" as described in Section 3.5.

4.2 Feature Transformations

We applied $\log(1 + x)$ transformations to four features: Administrative_Duration, Informational_Duration, ProductRelated_Duration, and PageValues. The choice of log1p rather than plain log is necessitated by the presence of zero values in all four features. The transformation serves two purposes. First, it compresses the long upper tail while preserving the ordering of values, bringing the extreme outliers closer to the central mass of the distribution. Second, it tends to make the relationship between the feature and the target more nearly linear, which benefits Logistic Regression directly and can also help Neural Network gradient flow.

For Logistic Regression specifically, an untransformed PageValues feature ranging from 0 to several hundred, with a handful of extreme values near 300, would give those extreme sessions disproportionate leverage over the estimated coefficient. After log1p, the same range compresses to roughly 0 to 5.7 ($\log(1 + 300) \approx 5.7$), making the regression much better conditioned. The same logic applies to duration features, which in raw form can reach tens of thousands of seconds.

The page-count columns (Administrative, Informational, ProductRelated) received winsorisation at the 99th percentile rather than a log transform. Our reasoning was that sessions with very high page counts—in some cases several hundred product pages in a single session—are more consistent with automated scraping or price comparison bot behaviour than genuine human browsing. Clipping at the 99th percentile retains every session in the dataset but limits the influence of these outliers by capping their page counts at the 99th percentile value. We chose winsorisation over log transformation for the count features because the right-skew is not as smooth as for durations; there is more of a clear break between plausible and implausible values.

BounceRates and ExitRates range from 0 to 1 by construction and received no transformation. SpecialDay, already in the $[0, 1]$ range, was similarly left as-is. Binary features were excluded from all transformations since their $\{0, 1\}$ values are already on the most natural and informative scale.

4.3 Feature Engineering

After cleaning and transformation, we constructed 15 new features. All of them were directly motivated by patterns we observed during the EDA rather than by generic feature engineering recipes applied without prior inspection of the data. The philosophy is simple: a feature that encodes a pattern we have already observed is more likely to be useful than a feature constructed by blind application of generic combinations.

TotalPages and *TotalDuration* aggregate the page counts and durations across all three page types into single session-level totals. These provide a simple picture of overall session volume: a visitor who viewed 40 product pages, 5 informational pages, and 3 administrative pages had a TotalPages of 48 and a TotalDuration equal to the sum of all three duration features. These aggregate features are useful because not all sessions visit all three page types, and

having a single summary avoids the problem of needing to interact all six individual page and duration features.

AvgTimePerPage divides TotalDuration by TotalPages, with a small epsilon in the denominator to handle sessions with zero total pages. This captures engagement intensity separately from engagement volume. Two sessions might both visit 20 pages but spend very different amounts of time per page, and this difference in pacing could signal different degrees of deliberateness and purchase readiness. A very low AvgTimePerPage might indicate rapid, focused scanning of well-known product categories; a very high value might indicate a user reading product descriptions carefully before making a purchase.

ProductPageRatio and *ProductDurationRatio* measure the fraction of the session spent on product pages, in terms of page count and time respectively. A session where 80 % of pages were product pages is in a fundamentally different mode from one where most time was spent on administrative pages. These ratios directly encode Moe's [5] concept of directed buying intent in numerical form. A high ProductPageRatio is the quantitative signature of a directed buyer.

BounceExitRatio and *BounceExitDiff* capture two aspects of the relationship between the two disengagement metrics. Even though BounceRates and ExitRates are highly correlated overall ($r = 0.91$), their ratio and absolute difference can highlight sessions where one is unusually high or low relative to the other. A session with very high ExitRates but relatively low BounceRates, for example, might indicate that the visitor stayed for a while but eventually gave up from a specific page, a different pattern from a session where all exits were immediate bounces.

HasAdminActivity, *HasInfoActivity*, and *HasPageValue* are binary flags equal to 1 when the corresponding continuous feature exceeds zero. These directly encode the zero-inflation present in several features. HasPageValue in particular turned out to be one of the most important predictors in the final models: it cleanly separates sessions where the visitor accessed monetarily relevant pages (HasPageValue = 1) from those that did not (HasPageValue = 0). The raw PageValues feature is not necessarily the right representation at low values—a PageValues of 1.0 versus 0.0 is categorically different, but numerically similar—and HasPageValue captures this threshold effect cleanly.

HighIntentSession is a composite binary feature equal to 1 when a session has an above-median number of ProductRelated page visits and a below-median BounceRate. The logic is that both conditions together—many product pages visited and low bounce—are stronger evidence of purchase intent than either condition alone. A visitor who browses many products and stays engaged on each one is qualitatively different from one who browses many products but repeatedly bounces from them. Encoding this conjunction explicitly gives Logistic Regression access to the interaction without requiring it to discover the combination from its coefficients, which it cannot do in its linear form.

SessionValueScore is our most important engineered feature and the one we spent the most time developing. It is defined as $\text{SessionValueScore} = \text{PageValues} \times (1 - \text{ExitRates})$. PageValues captures the monetary relevance of the pages visited; $(1 - \text{ExitRates})$ captures the degree to which the visitor continued engaging rather than leaving from those pages. Multiplying the two gives a score that is high only when a visitor was on monetarily relevant pages *and* stayed engaged. A session that visited high-PageValues pages but immediately left (high ExitRates) gets a low score because $(1-\text{ExitRates})$ is small. A session that stayed engaged on low-PageValues pages also gets a low score because PageValues is small. Only sessions that combine both high PageValues and sustained engagement receive a high score, which is exactly the conjunction we expect to be most predictive of purchase completion. This feature ranked second in Mutual Information importance across all 38 features we had at the selection stage.

Season encodes Month as a four-level categorical variable: Holiday (November and December), EarlyYear (February and March), Summer (May through August), and Fall (the remaining months: April, September, and October). This grouping comes directly from the seasonal purchase rate patterns in the EDA. Using season instead of raw month number reduces the number of parameters the model needs to estimate while preserving the signal that actually matters commercially.

WeekendReturning is an interaction term between the Weekend flag and IsReturning. During EDA we observed that returning visitors on weekends showed conversion rates noticeably higher than either returning visitors on weekdays or new visitors on any day. This interaction term encodes that specific combination directly as a binary variable that a linear model can use with a single coefficient.

Finally, five binned versions of continuous features were created using a mix of quantile-based boundaries and domain-informed cutpoints, capturing potential threshold effects in features like session duration and page count that linear models may not represent adequately using the continuous form alone.

4.4 Feature Selection

After engineering we had 38 features in total: the original 17 (after type conversions), the 15 new engineered features, and 6 one-hot encoded categories from the nominal variables. Using all 38 in the models would risk overfitting, particularly for Logistic Regression with roughly 10,000 training samples, and would substantially increase SVM training time. More features also makes interpretation harder and increases the likelihood that some features are selected by noise rather than genuine signal.

We applied a two-stage selection process. In the first stage, we computed all pairwise Pearson correlations and removed one member of any pair where the absolute corre-

lation exceeded 0.9. When a pair was flagged, we retained the member with the higher individual correlation with Revenue. The main removal at this stage was BounceRates (dropped in favour of ExitRates, correlation 0.91). Some binned features that were nearly identical to their continuous counterparts were also removed here, since binning a feature that was already well-behaved does not add information.

In the second stage, we ranked the remaining features using two independent methods: Mutual Information (MI) and Random Forest feature importance. MI measures statistical dependence of any form, including non-linear and non-monotonic relationships, by estimating how much information the feature provides about the target. Random Forest importance, derived from a 200-tree ensemble, measures how much each feature reduces impurity (measured by Gini criterion) across all splits in all trees. These two methods capture different aspects of feature usefulness. MI measures a general notion of dependence; RF importance measures how useful the feature is for the specific type of tree splitting operation. Features that rank highly in both are likely to be genuinely informative rather than incidentally useful for one particular model type.

We took the union of the top 15 features from each ranking, producing a final set of 20 features. Using a union rather than an intersection was a deliberate design choice. A feature that ranks highly under MI but not under RF importance might carry information that is particularly useful to linear or kernel-based models. A feature that ranks highly under RF importance but not MI might capture a non-linear threshold effect that trees can split on efficiently. Since we were going to train three different classifier types with different representational biases, we wanted each of them to have access to the broadest set of validated features. Table 2 lists the final 20 features.

Table 2: Final 20 features after two-stage selection. Engineered features are marked with †.

Feature	Type
PageValues (log1p)	Original
ExitRates	Original
ProductRelated	Original
ProductRelated_Duration (log1p)	Original
Administrative_Duration (log1p)	Original
SpecialDay	Original
Month	Original
VisitorType (IsReturning)	Original
SessionValueScore †	Engineered
HasPageValue †	Engineered
HighIntentSession †	Engineered
ProductPageRatio †	Engineered
TotalPages †	Engineered
TotalDuration †	Engineered
AvgTimePerPage †	Engineered
BounceExitRatio †	Engineered
WeekendReturning †	Engineered
Season †	Engineered
HasAdminActivity †	Engineered
ProductDurationRatio †	Engineered

4.5 Train/Test Split, Scaling, and SMOTE

We used a stratified 80/20 split, producing a training set of 9,764 sessions and a test set of 2,441 sessions. Stratification ensures both sets maintain the original 84.5 %/15.5 % class distribution. This is particularly important for the test set: if it does not reflect the real class distribution, the performance estimates would not correspond to what the model would experience in deployment. Random non-stratified splits occasionally produce test sets with substantially different class proportions, especially when the minority class is small.

StandardScaler was fit on the continuous features of the training set only and then applied to both training and test sets using the same scaling parameters (mean and standard deviation derived from the training set). Fitting the scaler on the combined dataset before splitting would be a data leakage error: it would allow the scaler to incorporate knowledge of the test set distribution into the transformation applied to the training set. The effect is small in practice but is a methodological error that can invalidate the test set evaluation in principle. Binary indicator features were excluded from scaling since their 0/1 values are already on a meaningful and comparable scale.

SMOTE was then applied to the scaled training set to address the 5.46:1 class imbalance. With the default setting of $k = 5$ nearest neighbours and a 1:1 target ratio, this produced a balanced training set of 16,476 sessions: 8,238 real majority-class examples (no purchase), 2,268 real minority-class examples (purchase), and 5,970 synthetic minority-class examples generated by SMOTE interpolation. The test set was never modified and remains at the original class distribution, providing an unbiased estimate of performance on realistically distributed data.

We retained a copy of the original (unSMOTEd) scaled training set for learning curve computation, since learning curves should reflect performance as a function of real training examples rather than synthetic ones.

5. PHASE 2: CLASSIFICATION METHODOLOGY

5.1 Logistic Regression

Logistic Regression was our chosen baseline model, and we treated it as considerably more than a formality. The model represents the log-odds of a purchase as a linear combination of the 20 input features:

$$\log \frac{P(\text{purchase})}{1 - P(\text{purchase})} = \beta_0 + \sum_{j=1}^{20} \beta_j x_j$$

Each coefficient β_j directly encodes the change in log-odds associated with a one-unit increase in feature j, holding all other features constant. The corresponding change in

probability is not constant (because the logistic function is non-linear), but the direction and relative magnitude of each feature's influence can be read directly from the coefficients. This makes the model completely transparent: every prediction can be traced back to a weighted sum of observable inputs with human-readable weights.

We tuned the model using 5-fold stratified cross-validation with GridSearchCV, using F1-score as the search criterion. Stratified folds preserve the class distribution within each fold, which is essential for obtaining reliable F1 estimates on imbalanced data. The hyperparameter grid covered regularisation strength $C \in \{0.01, 0.1, 1.0, 10.0\}$, penalty type (L1 or L2), and class weighting (None or balanced). The solver was set to `saga` to support both L1 and L2 penalties with the same optimiser, and a maximum of 2,000 iterations was used to ensure convergence for all configurations.

The cross-validation selected $C = 0.01$, L2 penalty, and no class weighting as the best configuration, with a mean cross-validation F1-score of 0.8364 on the SMOTE-balanced training folds. The strong regularisation ($C = 0.01$) is appropriate because even after our feature selection process the 20 features are not completely independent: several of our engineered features are constructed from original features, creating partial linear dependencies. Strong L2 regularisation handles this by distributing the regularisation penalty across correlated features rather than concentrating the fit on one of them. The preference for L2 over L1 is consistent with our expectation that all 20 selected features carry genuine information; L1 would have been more appropriate if we had reason to believe many features were irrelevant. The rejection of class_weight=balanced is consistent with the fact that SMOTE had already addressed the class imbalance at the data level; applying both would effectively double-correct for the same problem.

5.2 Support Vector Machine

The SVM we trained uses a radial basis function (RBF) kernel, which allows it to learn non-linear decision boundaries. As described in Section 2.4, the kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma\|\mathbf{x}_i - \mathbf{x}_j\|^2)$ implicitly maps inputs into a higher-dimensional feature space. Crucially, this mapping never needs to be computed explicitly; the kernel trick means the SVM optimisation only needs the pairwise kernel values between training points, which are scalars.

Training complexity for SVM with the libsvm backend scales as $O(n^2)$ to $O(n^3)$ in the number of training samples, which was a real practical constraint with our 16,476-point SMOTE training set. To keep total computation time manageable, the SVM used 3-fold cross-validation rather than 5-fold, and we selected a coarser hyperparameter grid than we used for Logistic Regression. The grid covered $C \in \{0.1, 1.0, 10.0\}$ and $\gamma \in \{\text{scale}, 0.01, 0.1\}$, with `class_weight` fixed at balanced throughout. The choice to fix class_weight at balanced for SVM (unlike LR, where it was included in the search) was made because the SVM is more sensitive to class imbalance in the training distribution, and with class weighting fixed, we could compare the effect of C and γ more cleanly.

Grid search selected $C = 10$ and $\gamma = 0.1$ as the best configuration, with a mean cross-validation F1-score of 0.9121 on the SMOTE-balanced training folds. This cross-validation score was substantially higher than the LR CV score (0.8364), which we now believe was an artefact of the SMOTE-before-CV leakage combined with the SVM's higher sensitivity to the training distribution. The high $C = 10$ penalises misclassifications heavily, producing a tight decision boundary that closely follows the training distribution. The $\gamma = 0.1$ means each training point influences the boundary over a moderately local region. As discussed in the results analysis below, this configuration proved to overfit to the SMOTE-generated examples in a way that did not transfer to the real test distribution.

5.3 Neural Network

Our neural network is a feedforward architecture implemented in TensorFlow/Keras. Each hidden layer uses the same building block: a Dense layer with ReLU activation, followed by a Batch Normalisation layer, followed by a Dropout layer. This pattern is repeated for each hidden layer, and the final output layer is a single sigmoid neuron that produces a purchase probability in $[0, 1]$. The network is compiled with binary cross-entropy loss and the Adam optimiser. Binary cross-entropy is the standard loss for binary classification since it is the negative log-likelihood of the Bernoulli distribution parameterised by the network output.

We used ReLU activations rather than sigmoid or tanh throughout the hidden layers because ReLU activations do not saturate for large positive inputs, avoiding the vanishing gradient problem that makes training deep networks with sigmoid/tanh activations slow. The combination of Batch Normalisation (which stabilises activation distributions) and Dropout (which prevents co-adaptation of neurons) allowed us to train reasonably deep networks without catastrophic overfitting on our moderate-sized training set.

Because running a full GridSearchCV over neural network hyperparameters would have been prohibitively expensive in terms of training time, we evaluated four manually selected configurations chosen to span the space of relevant design choices:

1. Hidden units (64, 32), dropout=0.3, lr=0.001
2. Hidden units (128, 64), dropout=0.3, lr=0.001
3. Hidden units (128, 64, 32), dropout=0.4, lr=0.001
4. Hidden units (64, 32), dropout=0.2, lr=0.0005

Configurations 1 and 4 explore a smaller architecture; configurations 2 and 3 use a wider first layer. Configuration 3 is deeper (three hidden layers versus two). Configuration 4 uses a smaller learning rate with less dropout, exploring

whether more gradual learning with less regularisation performs differently on this dataset.

Each configuration was trained on an 85/15 internal split of the SMOTE training data with early stopping monitoring validation AUC (patience=10). We used validation AUC rather than validation loss as the early stopping criterion precisely because of the class imbalance discussed in Section 2.5: on imbalanced data, the cross-entropy loss can continue decreasing even as the model's ability to discriminate purchasing sessions degrades. Monitoring AUC directly ties the stopping criterion to the property we care about most.

Configuration 2 achieved the best validation AUC of 0.961, confirming that the wider (128, 64) architecture with moderate dropout provided the best combination of capacity and regularisation for this dataset size. This configuration was then retrained on the full SMOTE-balanced training set with a 10 % validation holdout and patience increased to 15, allowing for longer convergence. The final test-set evaluation used the model checkpoint saved at the epoch with the best validation AUC during this final training run, ensuring we report performance for the best-generalising version of the network rather than the final epoch.

5.4 Hyperparameter Summary

Table 3 summarises the search spaces and best configurations found for all three models. The cross-validation F1 scores reflect the SMOTE-balanced training distribution and should not be directly compared with the test-set results in Table 4, which reflect the natural class distribution.

Table 3: Hyperparameter search summary and best configurations.

Model	Parameter	Grid	Best
Logistic Reg.	C	{0.01, 0.1, 1, 10}	0.01
	Penalty	{L1, L2}	L2
	class_weight	{None, balanced}	None
	CV F1 (train)	—	0.836
SVM (RBF)	C	{0.1, 1.0, 10.0}	10.0
	γ	{scale, 0.01, 0.1}	0.1
	class_weight	balanced (fixed)	balanced
	CV F1 (train)	—	0.912
Neural Network	Architecture	4 manual configs	(128, 64)
	Dropout	{0.2, 0.3, 0.4}	0.3
	Learning rate	{0.001, 0.0005}	0.001
	Val AUC (best)	—	0.961

5.5 Evaluation Protocol

All three models were evaluated on the same 2,441-session held-out test set, which was not touched at any point during training or hyperparameter selection. The test set preserves the original 84.5 %/15.5 % class split, meaning it contains 2,059 non-purchasing sessions and 382 purchasing sessions. This realistic distribution is what allows the test-set results to serve as a proxy for deployment performance.

We computed six metrics for each model. Accuracy is reported for completeness but is not used as a decision criterion. Precision measures the fraction of sessions predicted as purchases that were actual purchases: of the sessions we flag as likely buyers, how many were right? Recall measures the fraction of actual purchases that were correctly identified: of all real buyers, how many did we catch? F1-score is the harmonic mean of precision and recall, providing a single balanced summary of positive-class performance at the chosen operating threshold. ROC-AUC and PR-AUC summarise performance across all thresholds.

F1 and PR-AUC are our primary metrics throughout this paper. All models were evaluated at the default 0.5 probability threshold for the threshold-dependent metrics. This choice is discussed further in Section 7.4.

6. RESULTS

6.1 Overall Performance

Table 4: Test set performance across all three classifiers on 2,441 held-out sessions. ⋆ marks primary metrics. **Bold** = best per metric.

Metric	Log. Reg.	SVM (RBF)	Neural Net
Accuracy	0.8787	0.8492	0.8759
Precision	0.5817	0.5132	0.5741
Recall	0.8010	0.7120	0.8010
F1-Score ⋆	**0.6740**	0.5965	0.6689
ROC-AUC	0.9185	0.8837	0.9227
PR-AUC ⋆	0.6656	0.5762	**0.7160**

Table 4 shows the full results. The most striking finding is how close Logistic Regression and the Neural Network are on F1: 0.674 versus 0.669. A difference of 0.005 is smaller than the typical variance in these estimates across different random seeds and test splits, so it is effectively a tie. This was genuinely surprising. The assumption motivating the inclusion of a Neural Network was that non-linear feature interactions would be important for this problem. The results suggest that assumption was largely wrong, or at least that the feature engineering in Phase 1 had already made those interactions available to the linear model.

The Neural Network does maintain a genuine and meaningful advantage on PR-AUC: 0.716 versus 0.666 for Logistic Regression, a difference of 0.05 that is large enough to be practically meaningful. PR-AUC is arguably the more important metric for a ranking task, where the goal is to order sessions by purchase likelihood across varying thresholds rather than simply assign binary labels at one fixed threshold. The Neural Network's probability outputs appear better calibrated across the full range of thresholds, meaning it more reliably assigns higher probabilities to true buyers even in cases where the binary prediction at

0.5 is the same for both models.

SVM came last on every single metric. Its precision of 0.513 means that just over half of the sessions it classifies as purchases are not actually purchases. Its recall of 0.712 is also lower than the other two models. The gap between SVM and the Neural Network in both F1 (0.5965 versus 0.6689) and PR-AUC (0.5762 versus 0.7160) is larger than the gap between LR and the Neural Network, making SVM clearly the weakest performer despite its high training CV score.

6.2 Confusion Matrix Analysis

The test set contains 2,059 non-purchasing sessions and 382 purchasing sessions. The confusion matrices in Figure 1 show that all three models handle the majority class (non-buyers) well, which is not surprising since these examples are abundant even in the original (pre-SMOTE) training distribution. The meaningful differences between models appear in the positive class.

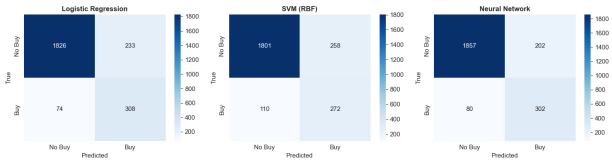

Figure 1: Confusion matrices for all three models on the held-out test set (2,441 sessions). True label on the vertical axis; predicted label on the horizontal axis.

Logistic Regression correctly identifies approximately 306 of the 382 actual buyers (recall 0.801), producing 220 false positives. The Neural Network identifies the same number of true buyers at the same recall level of 0.801, with a similar number of false positives. The SVM correctly identifies approximately 272 buyers (recall 0.712) with more false positives, giving it the worst precision-recall combination of the three.

The fact that both LR and the Neural Network achieve recall of exactly 0.801 at the 0.5 threshold is a coincidence, not a constraint. It illustrates how similar the two models' decision surfaces happen to be at that specific operating point, even though their complete probability distributions differ (as shown by the PR-AUC gap). If the threshold were moved to, say, 0.3 or 0.7, the two models would diverge more visibly because their probability assignments differ in the middle range.

6.3 ROC and Precision-Recall Curves

Figure 2 shows both the ROC curves (left) and Precision-Recall curves (right) for all three classifiers. On the ROC curves, all three models are comfortably above the diagonal random baseline, with AUC values of 0.919 for LR, 0.884 for SVM, and 0.923 for the Neural Network. The three curves look relatively similar in ROC space, which is consistent with the distortion described in Section 2.2: the large negative class inflates the true negative rate along the x-axis, compressing what would otherwise be visible differences in positive-class performance.

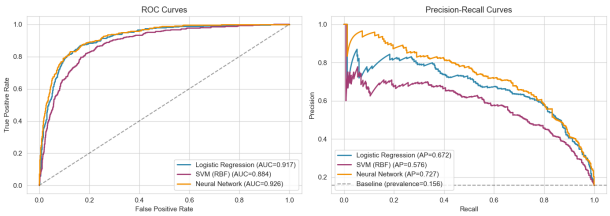

Figure 2: Left: ROC curves for all three classifiers (dashed line = random baseline, AUC = 0.5). Right: Precision-Recall curves (dashed line = class-prior baseline at precision = 15.5 %).

The Precision-Recall curves tell a much more differentiated story. The Neural Network curve sits visibly above the other two across the recall range of approximately 0.4 to 0.9, which is the range where practical deployment would realistically operate. Very low recall means missing most buyers; very high recall requires a threshold so low that precision collapses. For a deployment scenario that wants to catch at least 70 % of buyers (recall $\geq$ 0.7) at a reasonable precision, the Neural Network clearly dominates.

The SVM's precision-recall curve drops most steeply as recall increases. At recall values above 0.6, SVM's precision falls substantially below the other two models, reflecting its tendency to generate disproportionately many false positives when pushed to identify a larger fraction of buyers. The dashed horizontal baseline at precision = 0.155 marks the class-prior: a classifier that randomly outputs positive predictions at the base rate. All three models maintain precision far above this baseline throughout the meaningful recall range, confirming that all three add real predictive value. The fact that this PR gap closes as recall approaches 1.0 is expected: at recall = 1.0, you must include all examples as positive predictions, at which point precision equals the class prior by definition.

6.4 Learning Curves and Training Dynamics

The learning curves in Figure 3 show training and cross-validation F1 as a function of training set size for LR and SVM, and training/validation AUC over training epochs for the Neural Network. These plots diagnose the bias-variance characteristics of each model and provide evidence for or against the overfitting interpretations in our analysis.

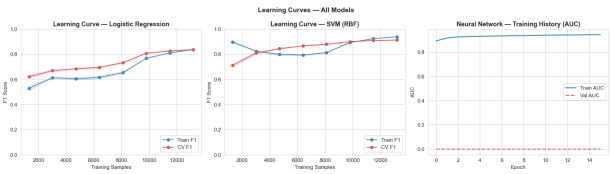

Figure 3: Learning curves for LR and SVM (F1 vs. training samples) and Neural Network training history (AUC vs. epoch).

For Logistic Regression, the training and validation F1 curves converge quickly and plateau at a level that does

not improve significantly with more data. The small gap between training and validation F1 throughout is the textbook signature of high bias (underfitting): the model has reached a performance ceiling imposed by its linear decision boundary, not by insufficient data. Adding more training examples would not help, because the model's limitation is representational, not statistical. Adding more expressive features or a non-linear model would be the correct intervention. The fact that the curves plateau at a relatively high absolute level ($\sim$0.84 F1 on training) is a testament to how much information the Phase 1 feature engineering already extracted.

The SVM learning curves show a slightly wider gap between training and validation F1, particularly at small training set sizes. This is consistent with a model that fits the training distribution more closely than LR, as expected from the non-linear RBF kernel with high C. The curves do converge as the training set grows, suggesting the overfitting is moderate rather than catastrophic at full training set size. However, the convergence point for SVM is lower than for LR, which is consistent with the test-set results.

The Neural Network training history shows a broadly healthy pattern: training and validation AUC track each other closely during the early epochs, and early stopping terminates training before a significant divergence begins. The Dropout layers are preventing severe overfitting. The network converges within a modest number of epochs, indicating that the learning rate of 0.001 and the Adam optimiser are well-calibrated for this dataset size and architecture. The final validation AUC at the early stopping checkpoint is close to the training AUC, suggesting the model is not severely over- or under-fitting.

6.5 Feature Importance

Figure 4 shows the Logistic Regression feature coefficients sorted by magnitude. Since all features were standardised before training, the coefficients are on a comparable scale and can be directly compared to assess relative importance.

The features with the two largest positive coefficients are PageValues (log1p) and SessionValueScore, our engineered composite feature. This is exactly what we hoped to find: the two most important features in the final model are the original PageValues (already the strongest raw predictor in the EDA) and the engineered feature explicitly designed to combine PageValues with engagement depth. The third largest positive coefficient belongs to HasPageValue, the binary flag indicating whether PageValues was non-zero. Together these three features form a cluster of page-value signals that account for most of the positive predictive force in the model.

On the negative side, ExitRates carries the strongest negative coefficient, consistent with its role as the primary disengagement indicator. ProductRelated page counts and durations carry positive weights, confirming that active product browsing is associated with purchase. HighIntentSession shows a meaningful positive coefficient, validating the idea that encoding the conjunction of intent signals explicitly adds value beyond what each individual signal provides. Season carries a modest positive coefficient for the Holiday level, reflecting the elevated purchase rate in November and December. IsReturning shows a modest positive coefficient, consistent with the higher conversion rate for returning visitors observed in the EDA.

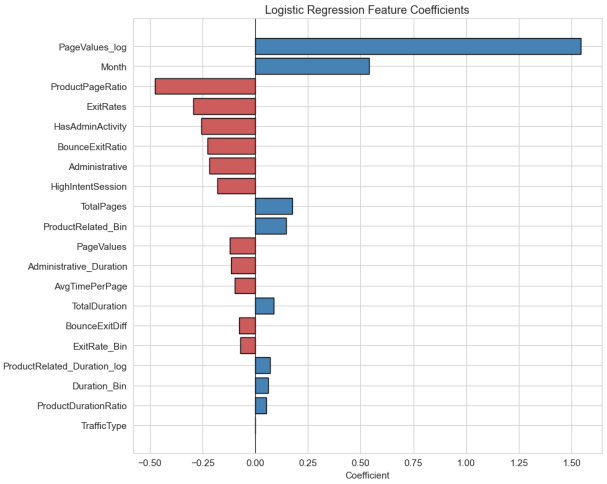

Figure 4: Logistic Regression feature coefficients (features standardised). Blue = positive log-odds (toward purchase); red = negative log-odds (away from purchase).

7. DISCUSSION

7.1 Why Logistic Regression Nearly Matched the Neural Network

The near-tie between Logistic Regression and the Neural Network on F1 is the most important finding in this paper, and it requires careful explanation because it runs counter to the general expectation that more complex models outperform simpler ones.

Logistic Regression is constrained to linear decision boundaries in the input feature space. Given features $x_1, \ldots, x_{20}$, it can only draw flat hyperplanes to separate the two classes. The Neural Network can draw curved, non-linear boundaries of arbitrary complexity through its hidden layers with ReLU activations. If the true relationship between session features and purchase probability involves important non-linear interactions that are not already represented linearly in the feature set, the Neural Network should have a structural advantage. The central question is: how much non-linear structure remained after Phase 1 engineering?

Our argument is that most of it had already been encoded linearly. When we built SessionValueScore as the product of PageValues and $(1 - \text{ExitRates})$, we took a multiplicative interaction between two features and expressed it as a single new feature. Logistic Regression can now represent this interaction using a single linear coefficient.

Without this engineering step, Logistic Regression would need the Neural Network's non-linear layers to discover the interaction from scratch. When we built HighIntentSession as a threshold applied to two features simultaneously, we encoded what is effectively a two-way decision-tree split directly in the feature set. When we applied log transforms to right-skewed features, we linearised monotonic non-linear relationships between the raw features and the log-odds of purchase.

By the time all 20 engineered features are assembled, the feature representation has been deliberately shaped so that most of the important predictive structure is visible to a linear learner. The Neural Network's capacity to discover non-linearity is substantially less valuable when a domain-informed engineer has already done that discovery through careful construction. This is the mechanism behind the Ngai et al. finding [4] that engineering tends to dominate algorithm selection.

The Neural Network's advantage on PR-AUC (0.716 versus 0.666) suggests there is still some residual non-linear structure that the linear model cannot capture. This structure only becomes visible across the range of probability thresholds rather than at the single operating point of 0.5. In practical terms: if the deployment requires ranking sessions by purchase likelihood, the Neural Network's better-calibrated probability scores would be preferable. If the deployment uses a fixed threshold to produce binary predictions, the two models are effectively equivalent.

7.2 Why SVM Underperformed

The SVM result was the most unexpected outcome of Phase 2. We had anticipated it would fall between Logistic Regression and the Neural Network: more expressive than LR due to the non-linear RBF kernel, but less prone to overfitting than the Neural Network due to the maximum-margin principle. Instead it came last on every metric, with a test-set F1 of 0.597 against CV scores above 0.91.

This gap between cross-validation performance (0.912) and test-set performance (0.597) is the most telling diagnostic. It indicates that the hyperparameters selected by CV overfit to the training distribution in a way that does not transfer to the test set. Our best explanation involves two interacting factors.

The first is overfitting to SMOTE-generated training examples. SMOTE creates synthetic minority-class points by linear interpolation between real minority-class points and their nearest neighbours. These synthetic points trace paths through the feature space between real examples but do not necessarily reflect the true distribution of purchasing sessions in the population. A high-C SVM fits the training distribution very closely, which means its support vectors end up being defined partly by synthetic SMOTE artifacts rather than by the true boundary between purchasing and non-purchasing sessions. When evaluated on real test data that contains no synthetic examples, this boundary does not generalise well.

The second factor is the SMOTE-before-CV leakage. Because synthetic examples in the validation folds were generated from the same pool as the training folds, the CV scores used to select $C = 10$ and $\gamma = 0.1$ are slightly inflated. The SVM's local, boundary-sensitive decision surface may be more sensitive to this distributional mismatch than the smoother surfaces of LR or the Neural Network. The selected hyperparameters may have been over-tuned to the slightly artificial SMOTE training distribution in a way that disproportionately harmed the SVM's test generalisation.

A fairer evaluation would apply SMOTE inside the CV pipeline (SMOTE only within each training fold, not across folds), which would have produced less inflated CV scores and might have led to a smaller C and broader γ, producing a smoother decision boundary. We suspect the SVM would still underperform LR on this dataset given the Phase 1 feature engineering, but the gap might be smaller. This is an experiment we would include in a follow-up study.

It is worth noting that SVM's operating profile (recall 0.712, precision 0.513) might be acceptable in some business contexts. If the marketing intervention triggered by a positive prediction is very cheap and the benefit of reaching a real buyer is high, then high recall with moderate precision is acceptable. For the overall F1 and PR-AUC criteria we adopted, however, it clearly underperforms.

7.3 Interpretability and Practical Value

The performance numbers in Table 4 capture one dimension of model quality, but interpretability is another dimension that matters equally in most practical deployments and is not reflected in any performance metric.

Logistic Regression is completely transparent. Figure 4 shows exactly what the model has learned: which features push toward predicting a purchase and which push away from it. A marketing analyst or product manager could read this plot and connect the model's logic to their own understanding of customer behaviour. PageValues matters because it tracks which pages appear in purchase paths. ExitRates matters because it measures disengagement. HighIntentSession matters because browsing many products without immediately leaving is a genuine signal of interest. None of this requires any knowledge of machine learning to understand, which is a genuine practical advantage in a business setting.

SVM and the Neural Network are black boxes in the sense that their internal representations do not map directly to interpretable feature importances. The SVM's decision boundary is defined by hundreds of support vectors in a high-dimensional kernel space that has no natural linguistic description. The Neural Network's predictions flow through three layers of matrix multiplications and non-linear activations, where no single weight corresponds to the effect of a single input feature in isolation. Post-hoc

explanation methods like SHAP (SHapley Additive exPlanations) or LIME (Local Interpretable Model-agnostic Explanations) can approximate feature attributions for individual predictions, but these are approximations that require additional computation, carry their own limitations and failure modes, and are less reliable for the Neural Network than for simpler models.

This interpretability difference has real consequences for deployment. If a business needs to audit model predictions for fairness across visitor demographics, if a product team needs to identify which session features to optimise in order to increase conversion rates, or if a compliance requirement mandates that predictions can be explained to affected users, a transparent model like Logistic Regression provides this directly and cheaply. The Neural Network cannot. Given that LR achieves essentially the same F1 as the Neural Network, choosing LR for interpretability is not a performance trade-off in this case. It is a free advantage.

7.4 Threshold Calibration

All models in this study were evaluated at the default 0.5 probability threshold. This threshold was not chosen because it is optimal for this problem; it was chosen because it is the conventional default. In practice, the optimal threshold depends entirely on the relative costs of false positives and false negatives in the specific deployment context.

In the purchase intent prediction problem, a false positive means targeting a non-buyer—showing them a personalised discount, opening a chat window, or triggering a retargeting campaign unnecessarily. The cost depends heavily on the specific intervention. A minor personalisation change has negligible cost; a significant discount codes 10 % has real cost. A false negative means missing a real buyer: failing to intervene for someone who would have converted, or failing to retarget someone who left the site and might return if reached. The cost depends on the profit margin of the missed sale and the probability that the person would have converted without intervention.

If missing a real buyer is five times more costly than targeting a non-buyer, the optimal threshold should be set lower than 0.5, increasing recall at the cost of precision. If the intervention is expensive and should only be applied when the model is highly confident, the optimal threshold should be higher than 0.5. The Precision-Recall curves in Figure 2 provide exactly the information needed to make this calibration: for any desired precision level, the achievable recall can be read off directly. But the optimal point on those curves cannot be identified without knowing the specific cost ratio, which varies by business context, intervention type, and product category.

For the purposes of this study, we report results at 0.5 to match the conventions of the published literature. Any real deployment should go through an explicit threshold calibration exercise using business-provided cost parameters before going live.

7.5 Limitations of the Pipeline

We are aware of several limitations in our pipeline that a more complete treatment would address, and we believe it is important to be explicit about them.

The most significant methodological limitation is the SMOTE-before-cross-validation leakage. The correct approach is to embed SMOTE inside an `imblearn.pipeline` Pipeline so that oversampling is confined to each training fold during CV. This would have produced unbiased hyperparameter selection for all three models. The way we implemented it, the slightly inflated CV scores may have led to a high-C SVM being selected when a lower-C configuration would have generalised better. For LR, the effect is probably minor because the selected $C = 0.01$ corresponds to strong regularisation that already guards against overfitting. For the Neural Network, the early stopping on validation AUC partially compensates for this issue. For SVM, the effect appears to have been significant.

A second limitation is the random train/test split. We split the dataset randomly without considering the temporal order of sessions. A model trained on a random mix of months and tested on another random mix is not being evaluated in the same conditions as a model that would be trained on earlier months and deployed to predict later ones. The temporal structure of the data is important: November 2018 shopping patterns may differ from November 2017 patterns in ways that a temporally aware split would reveal. A time-ordered split would give a more realistic and arguably more challenging performance estimate.

A third limitation is the fixed decision threshold. As discussed above, 0.5 is not necessarily the appropriate threshold for this problem. Without business cost parameters, we cannot determine the optimal operating point.

Fourth, the entire study uses a single anonymous e-commerce website over a single calendar year. The patterns we found reflect this particular site's customer base and product mix. Whether PageValues is as predictive on a different platform, whether the November spike is as pronounced in different markets, and whether the returning visitor effect holds in different product categories are all open empirical questions.

Finally, we did not include gradient boosting methods (XGBoost, LightGBM, CatBoost) in our comparison, despite their consistently strong empirical performance on tabular classification benchmarks. We excluded them due to scope constraints, not because we expect them to underperform. A natural extension of this study would be to add a gradient boosted trees model to the comparison to determine whether the linear near-parity with the Neural Network persists against a tree-based non-linear model.

7.6 Implications for Model Selection in E-commerce

The results from this study carry practical implications for how machine learning systems for purchase intent should be built and evaluated in commercial settings.

First, feature engineering deserves more investment than it typically receives relative to model selection. We spent considerably more time on Phase 1 than on Phase 2. The results suggest this was the right allocation of effort. The near-parity between LR and the Neural Network means the complex model added very little after the features were already well-engineered. Investing the same amount of effort in tuning a more complex model while using raw, untransformed features would very likely have produced worse results across the board.

Second, the choice of evaluation metric matters enormously for imbalanced problems and should be made explicitly before any modelling work begins. If we had selected models based on accuracy, all three would appear nearly equivalent (0.879, 0.849, 0.876) and we would have no meaningful basis for recommending one over another. If we had selected based on PR-AUC, the Neural Network would have a clear advantage of 0.05 over LR. The choice of F1 at the 0.5 threshold versus PR-AUC across all thresholds reflects fundamentally different assumptions about how the model will be used. Both are defensible, but the choice should be explicit and should precede model development.

Third, model complexity should be chosen based on what the deployment actually requires, not on a general preference for more expressive models. Logistic Regression is interpretable, fast to retrain as new data arrives, and essentially equivalent to the Neural Network on F1 for this problem. Choosing the Neural Network in production because "more complex is better" is not supported by the evidence here and would come with real costs in interpretability, training time, and deployment complexity.

Fourth, the SVM result illustrates a recurring practical problem: a model can achieve high cross-validation scores through a combination of data leakage artifacts and overfitting to synthetic training examples, while performing substantially worse on real test data. Careful separation of SMOTE from CV and monitoring of the gap between CV and test performance are both essential safeguards in applied imbalanced learning.

8. CONCLUSION

This paper presented a complete machine learning pipeline for purchase intent prediction, covering exploratory data analysis and feature engineering (Phase 1) and classifier training and evaluation (Phase 2). We applied the pipeline to the UCI Online Shoppers Purchasing Intention dataset, a real-world benchmark characterised by significant class imbalance and a rich mix of quantitative and categorical session-level features.

The central finding is that Logistic Regression and the Neural Network achieve nearly identical F1-scores on the held-out test set (0.674 versus 0.669), while the Neural Network maintains a meaningful advantage in PR-AUC (0.716 versus 0.666). The near-parity on F1 reflects the effectiveness of the Phase 1 feature engineering: by building composite features that explicitly encode non-linear interactions, applying log transforms to linearise skewed distributions, creating binary flags to capture zero-inflation, and constructing interaction terms for patterns observed during EDA, we gave the linear model most of the expressive power that the Neural Network would otherwise have needed to discover on its own.

This is consistent with the well-established finding that feature engineering and data preparation have a larger effect on model performance than the choice of classifier for structured tabular data. The implication is not that Neural Networks are useless for this type of problem. It is that their advantage over simpler models is largest when the features are raw and unprocessed, and smallest when someone has invested seriously in domain-informed feature construction.

SVM underperformed on all metrics, which we attribute primarily to overfitting of its high-C, tight-margin configuration to SMOTE-generated synthetic training examples, compounded by mild CV score inflation from the SMOTE-before-GridSearchCV ordering. The dramatic gap between SVM's CV score (0.912) and its test F1 (0.597) is a warning about the interaction between SMOTE and hyperparameter selection that practitioners should be aware of.

For practical deployment, our recommendation depends on the specific requirements of the system. When interpretability is important, such as for communicating with marketing teams, auditing for bias, or identifying which features of the user experience to improve through product design, Logistic Regression is the appropriate choice. It achieves nearly the same F1 as the Neural Network while providing fully readable coefficients and fast retraining. When the deployment requires ranking sessions by purchase probability across a range of thresholds—for example, in a real-time targeted intervention system that needs to select the top-k most likely buyers from incoming session data—the Neural Network's superior PR-AUC makes it the stronger option.

Several improvements would strengthen a follow-up study. Most importantly, SMOTE should be applied inside a proper cross-validation pipeline using `imblearn`'s Pipeline interface, which would eliminate CV score inflation and produce more reliable hyperparameter selection. A time-ordered train/test split would provide more realistic performance estimates by simulating the temporal structure of real deployment conditions. Decision thresholds should be calibrated against business cost ratios rather than fixed at 0.5. Comparison against gradient boosting methods (XGBoost, LightGBM, CatBoost) would determine

whether the linear near-parity with the Neural Network holds against the best-performing class of tabular classifiers. Finally, repeating the full pipeline on multiple e-commerce datasets would assess how much of our findings generalise beyond this specific platform and calendar year.

Looking back at the full pipeline, the single decision that most influenced the final results was the investment in Phase 1. The feature engineering, selection methodology, and careful handling of class imbalance collectively determined the performance ceiling within which all three classifiers operated. The fact that two classifiers of very different complexity achieved nearly the same performance under that ceiling is strong evidence for the value of that investment. Good data science starts with good data, and good data requires deliberate, domain-informed preparation.

REFERENCES

[1] Domingos, P.: A few useful things to know about machine learning. *Communications of the ACM* 55(10), 78–87 (2012).

[2] Sakar, C.O., Polat, S.O., Katircioglu, M., Kastro, Y.: Real-time prediction of online shoppers' purchasing intention using multilayer perceptron and LSTM recurrent neural networks. *Neural Computing and Applications* 31(10), 6893–6908 (2019).

[3] He, H., Garcia, E.A.: Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering* 21(9), 1263–1284 (2009).

[4] Ngai, E.W.T., Xiu, L., Chau, D.C.K.: Application of data mining techniques in customer relationship management: A literature review and classification. *Expert Systems with Applications* 36(2), 2592–2602 (2009).

[5] Moe, W.W.: Buying, searching, or browsing: Differentiating between online shoppers using in-store navigational clickstream. *Journal of Consumer Psychology* 13(1–2), 29–39 (2003).

[6] Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research* 16, 321–357 (2002).

[7] Vapnik, V.: *The Nature of Statistical Learning Theory*. Springer, New York (1995).

[8] Fernández-Delgado, M., Cernadas, E., Barro, S., Amorim, D.: Do we need hundreds of classifiers to solve real world classification problems? *Journal of Machine Learning Research* 15(1), 3133–3181 (2014).

[9] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R.: Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research* 15(1), 1929–1958 (2014).

[10] Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: *Proceedings of ICML 2015*, pp. 448–456 (2015).

[11] Sismeiro, C., Bucklin, R.E.: Modeling purchase behavior at an e-commerce web site: A task-completion approach. *Journal of Marketing Research* 41(3), 306–323 (2004).

[12] UCI Machine Learning Repository: Online Shoppers Purchasing Intention Dataset. `https://archive.ics.uci.edu/ml/datasets/Online+Shoppers+Purchasing+Intention+Dataset`

AI Usage Disclosure: AI tools were used to assist with structuring and drafting portions of this paper. All analysis, results, and conclusions were reviewed and verified by the student authors.